PROJECT ACTIVITY REPORT

Activity 2.2: Presence-Based Attendance Logic

Project: Multimodal Attendance & Engagement Agent

1. Activity Overview & Objective

This activity translates face-tracking data into meaningful attendance records. Instead of marking attendance from a single-frame detection, the system uses **presence-based logic**: attendance is confirmed only when a participant remains visible for a minimum duration during the session.

Fairness and false-positive mitigation. Presence duration is calculated from the timestamps collected during tracking. Intermittent appearances are accumulated, so brief or accidental detections do not produce false attendance, and temporary occlusions do not erase a student's progress. Once cumulative presence passes a predefined threshold, the participant is marked present.

Objectives

- Bind tracked faces to enrolled students through periodic face recognition.
- Accumulate presence time per student across tracking segments.
- Apply a minimum-duration rule to confirm attendance.
- Persist audit-ready logs with student ID, presence duration and confirmation status.

2. Presence Accumulation & Verification Pipeline

Attendance results are stored in structured logs containing participant identifiers and cumulative presence duration. This improves accuracy, reduces false positives and gives reliable records for classroom analytics and reporting.

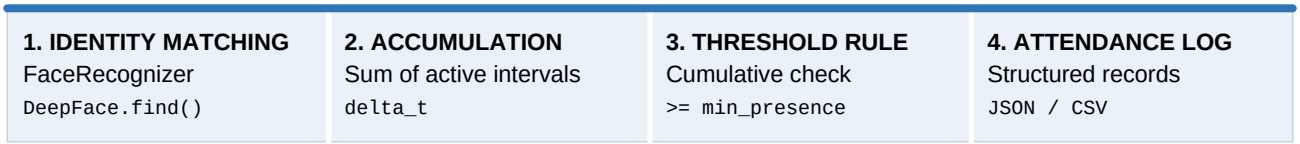

Figure 1: Presence accumulation and verification pipeline.

Verification stage	Mechanism & rule logic	Reliability & fairness benefit
Identity matching	Periodic facial verification against `student_db` via `DeepFace.find()`.	Associates tracked bounding-box trajectories with verified student identifiers.
Timestamp accumulation	Summation of active frame intervals (delta_t = t_current - t_last_seen) across tracking segments.	Accumulates intermittent visibility so temporary occlusions do not reset a student's progress.
Minimum duration threshold	Rule check: cumulative_duration >= min_presence_threshold.	Filters out transient passers-by, false-positive flashes and brief walk-ins.
Structured attendance logging	Persistence of verified student_id, presence_duration and confirmation status.	Produces audit-ready session logs for engagement analytics and institutional reporting.

3. Backend Implementation

To bind tracked presences to enrolled learners while keeping compute overhead manageable, `FaceRecognizer` (`backend/core/face_recognition.py`) uses DeepFace against a local reference database directory (`student_db`). Deep embedding extraction is compute-intensive, so recognition is invoked sparingly while the lightweight tracker supplies the timestamps used to accumulate verified presence.

3.1 Face Recognition Module

```
# File: backend/core/face_recognition.py
from deepface import DeepFace
import cv2
import numpy as np
import os


class FaceRecognizer:
    def __init__(self, db_path="student_db"):
        self.db_path = db_path
        if not os.path.exists(db_path):
            os.makedirs(db_path)

    def register_student(self, img_path, student_id):
        # In a real app, we'd store embeddings.
        # DeepFace.find uses a folder of images as the DB.
        pass

    def recognize(self, frame):
        try:
            # DeepFace accepts a numpy array directly.
            # DeepFace.find requires a db_path containing images;
            # with no DB yet, nothing can be recognized.
            if not os.listdir(self.db_path):
                return []

            # Compute-intensive: run sparingly
            dfs = DeepFace.find(
                img_path=frame,
                db_path=self.db_path,
                silent=True,
                enforce_detection=False
            )
            return dfs
        except Exception as e:
            print(f"Recognition error: {e}")
            return []
```

Student database layout. `DeepFace.find()` uses a folder of images as its database. A convenient convention, assumed by the code in this document, is one sub-folder per student, named with the student ID:

```
student_db/
|-- S101/
|   |-- front.jpg
|   `-- left.jpg
`-- S102/
    `-- front.jpg
```

Note. `register_student()` is a stub in the current implementation. DeepFace also caches embeddings in a pickle file inside `student_db`, which it refreshes when images are added; delete that cache if enrolment results look stale. The exact result columns returned by `find()` vary between DeepFace versions, so check the installed version before relying on specific column names.

4. Presence Accumulation Module

The recognizer says *who* a face is; the attendance manager decides *whether they were present long enough*. It maps tracker IDs to verified student IDs, adds `delta_t` for every frame in which a student is visible, and confirms attendance when the cumulative total reaches the threshold.

File: backend/core/attendance_manager.py (reference implementation)

```
# File: backend/core/attendance_manager.py
import csv
import json


class AttendanceManager:
    """Accumulates verified presence time per student and applies the rule
    cumulative_duration >= min_presence_threshold."""

    def __init__(self, min_presence_threshold=300.0, max_gap=2.0):
        self.min_presence_threshold = min_presence_threshold  # seconds
        self.max_gap = max_gap          # longer gaps are not counted as presence
        self.track_to_student = {}      # tracker ID -> verified student_id
        self.records = {}               # student_id -> presence record

    def bind(self, track_id, student_id):
        """Link a tracker ID to a student verified by face recognition."""
        self.track_to_student[track_id] = student_id

    def _update_student(self, student_id, ts):
        r = self.records.setdefault(student_id, {
            "student_id": student_id, "cumulative": 0.0, "last_seen": None,
            "first_seen": ts, "confirmed": False, "confirmed_at": None})
        if r["last_seen"] is not None:
            dt = ts - r["last_seen"]            # delta_t = t_current - t_last_seen
            if 0 < dt <= self.max_gap:
                r["cumulative"] += dt
        r["last_seen"] = ts
        if not r["confirmed"] and r["cumulative"] >= self.min_presence_threshold:
            r["confirmed"] = True
            r["confirmed_at"] = ts

    def update(self, tracked, timestamp):
        """tracked: telemetry list from CentroidTracker.update()."""
        seen = {self.track_to_student[t["id"]]
                for t in tracked if t["id"] in self.track_to_student}
        for student_id in seen:     # count each student once per frame
            self._update_student(student_id, timestamp)

    def report(self):
        return [{"student_id": r["student_id"],
                 "presence_duration": round(r["cumulative"], 2),
                 "status": "PRESENT" if r["confirmed"] else "NOT CONFIRMED",
                 "first_seen": r["first_seen"], "confirmed_at": r["confirmed_at"]}
                for r in self.records.values()]

    def save(self, path):
        rows = self.report()
        if path.endswith(".json"):
            with open(path, "w") as f:
                json.dump(rows, f, indent=2)
        else:
            with open(path, "w", newline="") as f:
                w = csv.DictWriter(f, fieldnames=list(rows[0]) if rows else [])
                w.writeheader()
                w.writerows(rows)
```

How the rule works

- **Accumulation:** each time a bound student is seen, `delta_t = t_current - t_last_seen` is added to the cumulative total.
- **Occlusion tolerance:** a gap longer than `max_gap` (camera dropout or leaving the room) adds nothing, but the existing total is kept, so progress is never reset.
- **Identity continuity:** if the tracker issues a new ID after a long absence, binding the new ID to the same student ID continues the same record.
- **Single confirmation:** the first time the total reaches the threshold, the student is marked PRESENT and the confirmation timestamp is stored.
- **One count per frame:** a student seen through two track IDs in the same frame is counted once.

Behaviour check

Tested with synthetic timestamps and a 10-second threshold: a student seen for 5 s, hidden for 4 s (longer than max_gap, so not counted), then seen for 5 s under a new tracker ID reached exactly 10.0 s and was marked PRESENT; a second student seen for only 3 frames stayed NOT CONFIRMED. The CSV and JSON exports were written correctly. This used synthetic data only; it has not been run with real video or DeepFace.

5. Integrating Recognition with Tracking

Recognition runs only for tracks that are not yet bound to a student, at most once per `interval` seconds per track. Each face is cropped from the tracker's bounding box so DeepFace analyses one face instead of the whole frame.

Added to backend/core/face_recognition.py

```
# Additional helper for FaceRecognizer
def student_id_from_result(self, dfs):
    # Assumes student_db/<student_id>/<image>.jpg layout
    if dfs and len(dfs[0]) > 0:
        return os.path.basename(os.path.dirname(dfs[0].iloc[0]["identity"]))
    return None
```

Wiring recognizer, tracker and attendance manager

```
# File: scripts/run_attendance.py (core loop; unchanged parts omitted)
import time
from backend.core.face_recognition import FaceRecognizer
from backend.core.attendance_manager import AttendanceManager

recognizer = FaceRecognizer("student_db")
attendance = AttendanceManager(min_presence_threshold=300, max_gap=2.0)
last_try, RECOGNIZE_EVERY = {}, 5.0          # seconds per unbound track

def process(frame, tracked):
    now = time.time()
    for t in tracked:
        tid = t["id"]
        if tid in attendance.track_to_student:
            continue                          # already identified
        if now - last_try.get(tid, 0) < RECOGNIZE_EVERY:
            continue                          # throttle expensive call
        last_try[tid] = now
        x, y, w, h = t["bbox"]
        face = frame[max(y, 0):y + h, max(x, 0):x + w]
        student_id = recognizer.student_id_from_result(recognizer.recognize(face))
        if student_id:
            attendance.bind(tid, student_id)
    attendance.update(tracked, now)           # cheap: runs every frame

# At session end:  attendance.save("attendance_session_01.csv")
```

6. Configuration Reference

Parameter	Example	Meaning	Guidance
min_presence_threshold	300 s	Cumulative visible time required to mark PRESENT.	Set per class length, for example a fraction of the session.
max_gap	2.0 s	Largest frame-to-frame gap still counted as continuous presence.	Raise for low frame rates; lower to be stricter.
RECOGNIZE_EVERY	5 s	Minimum time between recognition attempts per unbound track.	Raise if frame rate drops; lower for faster identification of arrivals.
db_path	student_db	Folder of enrolled student images.	Use several images per student (different angles and lighting).
enforce_detection	False	Avoids exceptions when DeepFace finds no face.	Keep False for live video with occlusions.

7. Sample Attendance Log

Each session produces one record per recognized student:

```
student_id,presence_duration,status,first_seen,confirmed_at
S101,312.4,PRESENT,1730000000.0,1730000303.8
S102,87.1,NOT CONFIRMED,1730000150.2,
```

The sample rows are illustrative values showing the output format, not real results.

8. Key Engineering & Architectural Highlights

• **Decoupled tracking and recognition cadence:** since `DeepFace.find()` is expensive, recognition runs sparingly while lightweight frame-to-frame tracking accumulates time, preserving real-time frame rates.

• **Graceful empty-database guardrails:** `if not os.listdir(self.db_path): return []` and `enforce_detection=False` prevent crashes when starting a new cohort or processing partially occluded frames.

• **Cumulative fairness guarantee:** aggregating intermittent appearances into one duration metric evaluates students across the whole session instead of penalizing momentary posture shifts or camera dropouts.

9. Known Limitations & Future Improvements

Limitation	Impact	Possible improvement
register_student() is a stub	Enrolment is manual (copying images into student_db).	Implement enrolment that saves images and refreshes embeddings.
A wrong match binds the whole track to the wrong student	Presence is credited to the wrong person.	Require several consistent matches (majority vote) before binding.
Similar-looking faces or poor images	Misidentification or missed recognition.	Use multiple enrolment images, a distance cut-off, and review unmatched faces.
Time-based confirmation only	A student visible but not engaged is still counted.	Combine with engagement signals in the multimodal fusion layer.
Face images are personal data	Privacy and consent obligations.	Obtain consent, restrict access to student_db, and define retention rules.

10. Validation Checklist

Tick each item after running it on your machine:

Check	How to verify	Status
Empty student_db handled	recognize() returns [] without an exception	[ ]
Enrolled student recognized	Known face returns the correct student ID	[ ]
Passer-by not confirmed	Short appearance stays NOT CONFIRMED	[ ]
Occlusion does not reset time	Cover face briefly; cumulative total is kept	[ ]
Threshold triggers PRESENT	Status changes once the total reaches the threshold	[ ]
Log file written	CSV/JSON contains student_id, presence_duration, status	[ ]